

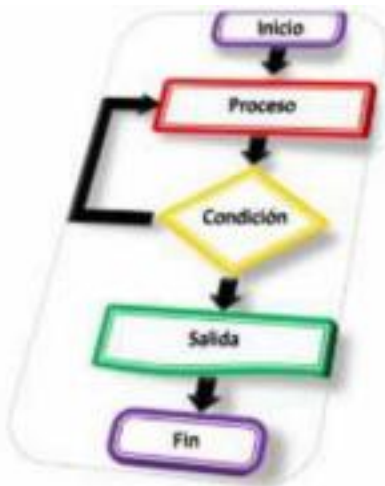


PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRALFORMATO GUÍA DE APRENDIZAJE

Guía 3 Lógica de programación y Algoritmia

1. IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- Denominación del Programa de Formación: Técnico en Programación de Software. • Código del Programa de Formación: 233104
- Nombre del Proyecto: desarrollo de aplicaciones de software para el sector empresarial • Fase del Proyecto, Fase 3: Ejecución del Proyecto (Ejecución)
- Actividad de Proyecto: Actividad 5: construir interfaces y procesos lógicos de acuerdo con las necesidades del cliente.
- Competencias:
Técnicas:
 - 220501096 - Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.
 - Resultados de Aprendizaje Alcanzar:
 - 2584211 - Resolver procesos lógicos a través de la implementación de algoritmos y el lenguaje de programación seleccionado.
 - Duración de la Guía: 100 horas (80 horas presenciales y 10 horas desescolarizadas)



2. PRESENTACIÓN

Toda persona que se inicie en el mundo de la programación necesariamente debe tener en cuenta algunos elementos esenciales para llegar a ser un buen programador. La lógica es uno de estos elementos. Ésta es esencial en el conocimiento de la programación, porque es

Esta guía le permitirá al aprendiz introducirse en el mundo de la programación a través de una serie de conocimientos que les darán las herramientas necesarias para obtener los procesos mentales para lograr determinado objetivo.

Es de vital importancia que los aprendices realicen las actividades propuestas con conciencia, ya que esto les permitirá adquirir los conocimientos y la lógica necesaria de manera efectiva. Los programadores no se hacen de la noche a la mañana, es un proceso que se puede ir realizando poco a poco y la lógica es uno de los

aspectos que primero se deben desarrollar. Eso sí, hay que tener algunos conocimientos previos, por ejemplo, manejo básico del computador y fundamentos de matemáticas, entre otros, también hay que tener mucha curiosidad y práctica constante. **3.**

FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE



3.1- Lógica Booleana con Tablas de verdad, estructuras secuenciales, de selección e iteración.

Tiempo estimado: 5 horas

Parte 1:

La lógica Booleana es un mundo binario de 1 y 0 donde las compuertas y tablas de verdad toman relevancia a continuación se muestra un pequeño resumen de lo visto en clase:

NOMBRE	TABLA DE VERDAD	CIRCUITO	OPERACIÓN															
AND	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1	74LS08 	$F = AB$
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
NAND	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0	74LS00 	$F = \overline{AB}$ $= \overline{A+B}$
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
OR	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1	74LS32 	$F = A+B$
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOR	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0	74LS02 	$F = \overline{A+B}$ $= \overline{A} \cdot \overline{B}$
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0	74LS86 	$F = A \oplus B$ $= \overline{A}B + A\overline{B}$
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
NOT	<table><tr><td>A</td><td>F</td></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0	74LS04 	$F = \overline{A}$									
A	F																	
0	1																	
1	0																	

Teniendo presente los conocimientos actualmente adquiridos en la materia y la explicación dada en clases anteriores sobre compuertas lógicas:

- 1- Elabore un diseño desde su perspectiva del circuito lógico que tendría un semáforo por medio de compuertas digitales con su paso a paso y explicación.



- 2- Analice y observe la siguiente ecuación:

$$X = [A\bar{B}((C + BD) + \bar{A}\bar{B})]C$$

Obtener lo siguiente:

- Tabla de verdad a partir de la ecuación.
- Dibujo del Circuito Lógico a partir de las compuertas vistas en clase.

3- Analice y observe la siguiente ecuación:

$$X = A\bar{B}C[BD + CDE] + AC$$

S

Obtener lo siguiente:

- Tabla de verdad a partir de la ecuación.
- Dibujo del Circuito Lógico a partir de las compuertas vistas en clase.

Nota: Si desea probar de una manera didáctica el funcionamiento de sus circuitos, puede utilizar el simulador de tinkercad y probar el desarrollo de su circuito lógico expuesto en la guía. El enlace del software es: <https://www.tinkercad.com/>

Parte 2:

En esta parte el estudiante reforzará los conceptos vistos en clase sobre Python, streamlit y la lógica de programación aprendida de momento teniendo presente los siguientes ejercicios:



1- Desarrollar un diagrama de flujo y diseñar un programa web que sea desplegado en streamlit orientado al cálculo del área y perímetro de las siguientes figuras:

- Rectángulo, cuadrado, triángulo, círculo y de un paralelogramo.

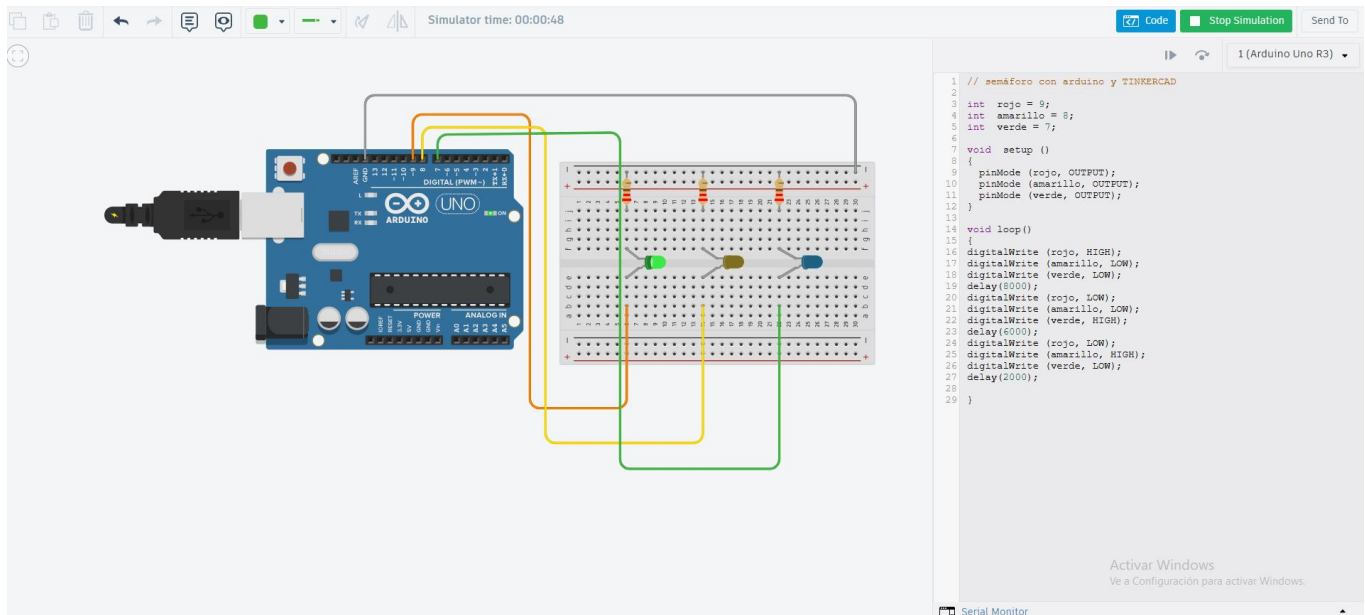
El programa en streamlit debe mostrar la figura del rectángulo, cuadrado, triángulo, círculo y del paralelogramo con la ecuación respectiva de cada cálculo.

- 2- ¿Cómo sé que un número es par e impar?, además ¿Cómo sé que ese número es primo o no lo es? Y ¿cómo sé que ese número es divisible por otro número? Desarrolle un diagrama de flujo y un programa en Python que resuelva estas preguntas.
- 3- La programación va mucho más allá de las páginas web, observe el siguiente repositorio, investigue que es YOLO y pruebe el algoritmo. Describa en palabras sencillas que entiende por la arquitectura YOLO y plantee una solución en la que pueda usar el software. El repositorio es el siguiente:
[https://github.com/dialejobv/U_Militar/blob/main/2\)%20Yolo/Explicaci%C3%B3n_Arq_YOLO.md](https://github.com/dialejobv/U_Militar/blob/main/2)%20Yolo/Explicaci%C3%B3n_Arq_YOLO.md)

Parte 3

Teniendo presente lo expuesto en la clase desarrolle una página web con Python y streamlit del proyecto que venía desarrollando con html, css y javascript. Explique las diferencias del desarrollo con Python y streamlit, anexo a lo anterior anexe un chatbot de apoyo a su página web e ingrese diferentes tablas, gráficos y estilos.

Desarrollar un informe en pdf, explicar el desarrollo en un readme.md y anexar todo al repositorio de github, posteriormente anexar el enlace en la actividad 5 del google classroom.



El algoritmo You Only Look Once (YOLO), es un sistema de código abierto para [detección de objetos](#) en tiempo real, el cual hace uso de una única red neuronal convolucional para detectar objetos en imágenes. Para su funcionamiento, la [red neuronal](#) divide la imagen en regiones, prediciendo cuadros de identificación y probabilidades por cada región; las cajas son ponderadas a partir de las probabilidades predichas. El algoritmo aprende representaciones generalizables de los objetos, permitiendo un bajo error de detección para entradas nuevas, diferentes al conjunto de datos de entrenamiento.¹

Buttes, West Placer y Yolo. La primera misa en el norte de California se celebró en una casa particular de Sacramento el 10 de agosto de 1850 por el misionero...

13 kB (972 palabras) – 16:08 14 may 2026

1886, mediante el breve Divinitus nobis del papa León XIII, la diócesis amplió considerablemente su territorio con los condados de Yolo, El Dorado, Amador...

Versión de YOLO	Nano (n)	Small (s)	Medium (m)	Large (l)	X-Large (x)	Características Clave
YOLOv5 (Refresh)	28.0%	37.4%	45.4%	49.0%	50.7%	Introdujo un despliegue masivo y sencillo en Python/PyTorch.
YOLOv8	37.3%	44.9%	50.2%	52.9%	53.9%	Arquitectura sin anclas (anchor-free) y soporte nativo multitarea.
YOLOv9	—	46.8%	51.4%	—	53.0% (c) / 55.6% (e)	Enfoque en información de gradiente programable (PGI) para máxima fidelidad.
YOLOv10	38.5%	46.3%	51.3%	53.4%	54.4%	Eliminación de NMS (Non-Maximum Suppression) para reducir latencia.

YOLO11	39.5%	47.0%	51.5%	53.4%	54.7%	Mayor eficiencia de parámetros (-22%) con mejoras en precisión y velocidad.
YOLO26	42.1%	49.2%	53.1%	54.7%	55.8%	El estándar más moderno; totalmente NMS-free y optimizado para arquitecturas de borde.

abla Comparativa de Arquitecturas de Visión Artificial

Arquitectura	Enfoque	Velocidad	Precisión	Uso Típico
R-CNN, Fast R-CNN, Faster R-CNN	Detectores de dos etapas. Primero proponen regiones candidatas (donde podría haber objetos) y luego clasifican cada región.	Lento (no es en tiempo real)	Muy Alta	Aplicaciones donde la precisión es crítica y la velocidad no es la principal preocupación, como en imágenes médicas.
YOLO (y SSD)	Detectores de una etapa. Realizan la detección y clasificación en una sola pasada.	Muy Rápida (tiempo real)	Alta	Aplicaciones que requieren velocidad, como coches autónomos, videovigilancia, robótica.
RT-DETR	Detector de una etapa basado en Transformers. Utiliza transformadores de visión para entender el contexto global de la imagen.	Rápida	Muy Alta	Similar a YOLO, pero con un enfoque diferente que puede ser superior en ciertos escenarios con objetos muy superpuestos.
EfficientDet	Enfoque en la eficiencia. Utiliza un escalado compuesto para optimizar el rendimiento con menos recursos computacionales.	Variable	Alta	Aplicaciones en dispositivos con recursos limitados (edge devices).
SAM (Segment Anything Model)	Segmentación. A diferencia de YOLO, que detecta objetos con recuadros, SAM se centra en la segmentación a nivel de píxel, aislando el contorno exacto de los objetos.	Lenta (por ahora)	Muy Alta	Etiquetado de datos, análisis de imágenes complejas donde se necesita el contorno exacto.
Mask R-CNN	Extensión de Faster R-CNN. Detecta objetos con recuadros y, simultáneamente, genera una máscara de	Lenta	Muy Alta	Aplicaciones que requieren tanto la detección como el contorno exacto, como en la industria automotriz

1. Declaración de Variables
 Al inicio del código se definen tres variables de tipo entero (`int`) para

asignar los pines digitales del Arduino a cada color de LED:

segmentación para cada objeto.

o la imagen médica.

- `rojo = 13` (Nota: Aunque la variable se llama "rojo", en el circuito físico de la protoboard está conectado un LED verde).
- `amarillo = 12`
- `verde = 11` (Conectado al LED azul en el circuito).

2. Configuración Inicial

(`void setup`)

Esta función se ejecuta una sola vez al encender el Arduino. Utiliza la instrucción `pinMode()` para configurar los tres pines como **salidas digitales** (`OUTPUT`). Esto le dice a la placa que enviará energía a través de esos pines para encender los componentes.

3. Bucle Principal (`void loop`)

Esta sección se repite infinitamente y contiene la secuencia de encendido y apagado utilizando dos instrucciones clave:

- `digitalWrite(pin, HIGH / LOW):` `HIGH` envía energía (5V) para encender el LED y `LOW` corta la energía (0V) para apagarlo.
- `delay(tiempo):` Pausa el programa durante el tiempo indicado en milisegundos (1000 milisegundos = 1 segundo).

La secuencia exacta que realiza es la siguiente:

1. **Enciende el primer LED** (`rojo`), asegura que los otros dos estén apagados y espera **2 segundos** (`delay(2000)`).
2. **Apaga el primer LED y enciende el segundo** (`amarillo`), manteniendo el

tercero apagado durante **0.5 segundos** (`delay(500)`).

3. **Apaga el segundo LED y enciende el tercero** (`verde`), manteniendo el primero apagado durante **2 segundos** (`delay(2000)`).

4. **Apaga el tercer LED y enciende el segundo** (`amarillo`) durante **0.5 segundos** (`delay(500)`).

Al terminar el último paso, el ciclo vuelve a comenzar desde el principio de forma automática.

¿Te gustaría saber cómo **corregir los nombres de las variables** para que coincidan exactamente con los colores de los LEDs del circuito, o necesitas ayuda para **modificar los tiempos de espera**?

N

```
class JugadorFutbol:
    def __init__(self, nombre, equipo, media, ritmo, tiro, pase, fisico):
        self.nombre = nombre
        self.equipo = equipo
        self.media = media
        self.stats = {
            "Ritmo": ritmo,
            "Tiro": tiro,
            "Pase": pase,
            "Fisico": fisico
        }

    def celebrar_gol(self):
        print(f"{self.nombre} ha marcado un golazo!")
        print("!!! GOOOOOOOOOOL !!! ⚽🐾")

# Creando la tarjeta de Lionel Messi con sus datos
messi = JugadorFutbol(
    nombre="Lionel Messi",
    equipo="Inter Miami",
    media=99,
    ritmo=92,
    tiro=99,
    pase=99,
    fisico=85
```



1. Declaración de Variables

Al inicio del código se definen tres variables de tipo entero (int) para asignar los pines digitales del Arduino a cada color de LED:

- rojo = 13 (Nota: Aunque la variable se llama "rojo", en el circuito físico de la protoboard está conectado un LED verde).
- amarillo = 12
- verde = 11 (Conectado al LED azul en el circuito).

2. Configuración Inicial (void setup)

Esta función se ejecuta una sola vez al encender el Arduino. Utiliza la instrucción `pinMode()` para configurar los tres pines como salidas digitales (OUTPUT). Esto le dice a la placa que enviará energía a través de esos pines para encender los componentes.

3. Bucle Principal (void loop)

Esta sección se repite infinitamente y contiene la secuencia de encendido y apagado utilizando dos instrucciones clave:

- *`digitalWrite(pin, HIGH / LOW)`: HIGH envía energía (5V) para encender el LED y LOW corta la energía (0V) para apagarlo.*
- *`delay(tiempo)`: Pausa el programa durante el tiempo indicado en milisegundos (1000 milisegundos = 1 segundo).*

La secuencia exacta que realiza es la siguiente:

1. Enciende el primer LED (rojo), asegura que los otros dos estén apagados y espera 2 segundos (delay(2000)).
2. Apaga el primer LED y enciende el segundo (amarillo), manteniendo el tercero apagado durante 0.5 segundos (delay(500)).
3. Apaga el segundo LED y enciende el tercero (verde), manteniendo el primero apagado durante 2 segundos (delay(2000)).
4. Apaga el tercer LED y enciende el segundo (amarillo) durante 0.5 segundos (delay(500)).

Al terminar el último paso, el ciclo vuelve a comenzar desde el principio de forma automática.

¿Te gustaría saber cómo corregir los nombres de las variables para que coincidan exactamente con los colores de los LEDs del circuito, o necesitas ayuda para modificar los tiempos de espera?

Expresión original
$$X = [A \neq B]((C + BD) + [A \neq B])C$$
 2. Distribuir el término $[A \neq B]$ dentro del corchete Multiplicamos $[A \neq B]$ por cada uno de los dos sumandos que están dentro del paréntesis exterior:
$$X = [A \neq B](C + BD) + [A \neq B][A \neq B]C$$
 3. Simplificar el 1. Expresión original

Paso	Expresión	Ley / Regla Aplicada	Explicación
1	$[A\bar{B}((C + BD) + \bar{A}\bar{B})]C$	Expresión Original	Estado inicial del problema.
2	$[A\bar{B}(C + BD) + A\bar{B}\bar{A}\bar{B}]C$	Propiedad Distributiva	Se multiplica $A\bar{B}$ por los dos términos de adentro.
3	$[A\bar{B}(C + BD) + 0]C$	Ley del Complemento	Como $A \cdot \bar{A} = 0$, todo ese término se vuelve 0.
4	$[A\bar{B}(C + BD)]C$	Identidad / Elemento Neutro	Sumar cero no altera la expresión ($Y + 0 = Y$).
5	$A\bar{B}C(C + BD)$	Propiedad Conmutativa	Se mete la C del final y se reordena junto a las variables.
6	$(A\bar{B}C \cdot C) + (A\bar{B}C \cdot BD)$	Propiedad Distributiva	Se multiplica $A\bar{B}C$ por cada elemento dentro del paréntesis.
7	$A\bar{B}C + 0$	Idempotencia e Inverso	En la izquierda $C \cdot C = C$. En la derecha $\bar{B} \cdot B = 0$.
8	$A\bar{B}C$	Resultado Simplificado	Se elimina el +0 final para obtener la mínima expresión.

1	$A\bar{B}C[BD + CDE] + AC$	Expresión Original	Estado inicial del problema.
2	$(A\bar{B}C \cdot BD) + (A\bar{B}C \cdot CDE) + AC$	Propiedad Distributiva	Se multiplica el término exterior $A\bar{B}C$ por los dos sumandos dentro del corchete.
3	$(A(\bar{B}B)CD) + (A\bar{B}(CC)DE) + AC$	Propiedad Conmutativa	Se reordenan las variables idénticas o complementarias juntas para identificarlas fácilmente.
4	$(A \cdot 0 \cdot CD) + (A\bar{B}CDE) + AC$	Leyes de Complemento e Idempotencia	En el primer término: $\bar{B}B = 0$. En el segundo término: $CC = C$.
5	$0 + A\bar{B}CDE + AC$	Elemento Absorbente	Cualquier expresión multiplicada por cero da cero ($Y \cdot 0 = 0$).
6	$A\bar{B}CDE + AC$	Identidad / Elemento Neutro	El cero sumado no altera la expresión ($Y + 0 = Y$).
7	$AC(A\bar{B}DE + 1)$	Factor Común (Distribución Inversa)	Se extrae el término común AC de ambos sumandos.
8	$AC \cdot 1$	Ley de Dominación	Cualquier variable o expresión sumada con 1 es igual a 1 ($Y + 1 = 1$).
9	AC	Resultado Simplificado	Cualquier término multiplicado por 1 mantiene su valor ($Y \cdot 1 = Y$).